

PYTHON SET-2

SCENARIO BASED QUESTIONS

1. Scenario: You are developing a banking application that categorise transactions based on the amount entered.

Write logic to determine whether the amount is positive, negative, or zero.

Logic:

1. Take a number as input.
2. Check if the number is greater than 0.
3. If yes, the number is positive.
4. Otherwise, check if the number is less than 0.
5. If yes, the number is negative.
6. If neither condition is true, the number is zero.

2. Scenario: A digital locker requires users to enter a numerical passcode. As part of a security feature, the system checks the sum of the digits of the passcode.

Write logic to compute the sum of the digits of a given number.

Logic:

1. Take a number as input.
2. Create a variable to store the sum and initially set it to 0.
3. Take the last digit of the number.
4. Add that digit to the sum.
5. Remove the last digit from the number.
6. Repeat steps 3–5 until no digits remain.
7. Return the final sum.

3. Scenario: A mobile payment app uses a simple checksum validation where reversing a transaction ID helps detect fraud.

Write logic to take a number and return its reverse.

Logic:

1. Take a number as input.
2. Create an empty value to store the reversed number.
3. Take the last digit of the number.
4. Add that digit to the reversed number.
5. Remove the last digit from the original number.
6. Repeat until all digits have been processed.
7. Return the reversed number.

4. Scenario: In a secure login system, certain features are enabled only for users with prime-numbered user IDs.

Write logic to check if a given number is prime.

Logic:

1. Take a number as input.
2. If the number is less than 2, it is not prime.
3. Check whether the number can be divided evenly by any number from 2 up to its square root.
4. If it is divisible by any of them, it is not prime.
5. If no number divides it evenly, it is prime.

5. Scenario: A scientist is working on permutations and needs to calculate the factorial of numbers frequently.

Write logic to find the factorial of a given number using recursion.

Logic:

1. Take a non-negative number as input.
2. If the number is 0 or 1, return 1. This is the stopping condition.

3. Otherwise, multiply the number by the factorial of the number immediately before it.
4. Continue calling the same process with the smaller number.
5. When 0 or 1 is reached, return 1.
6. The results are multiplied while returning back.
7. Return the final factorial value.

6. Scenario: A unique lottery system assigns ticket numbers where only Armstrong numbers win the jackpot.

Write logic to check whether a given number is an Armstrong number.

Logic:

1. Take a number as input.
2. Count the number of digits in the number.
3. Separate each digit.
4. Raise each digit to the power of the total number of digits.
5. Add all the resulting values.
6. Compare the calculated sum with the original number.
7. If both are equal, the number is an Armstrong number.
8. Otherwise, it is not an Armstrong number.

7. Scenario: A password manager needs to strengthen weak passwords by swapping the first and last characters of user-generated passwords.

Write logic to perform this operation on a given string.

Logic:

1. Take a string/password as input.
2. Identify the first character.
3. Identify the last character.
4. Keep all the middle characters unchanged.
5. Replace the first character with the last character.
6. Replace the last character with the first character.
7. Return the modified string.

8.Scenario: A low-level networking application requires decimal numbers to be converted into binary format before transmission.

Write logic to convert a given decimal number into its binary equivalent.

Logic:

1. Take a decimal number as input.
2. Divide the number by 2.
3. Record the remainder.
4. Take the quotient and divide it by 2 again.
5. Continue until the quotient becomes 0.
6. Collect all the remainders.
7. Read the remainders from the last one to the first one.
8. The resulting sequence is the binary equivalent.

9. Scenario: A text-processing tool helps summarize articles by identifying the most significant words.

Write logic to find the longest word in a sentence.

Logic:

1. Take a sentence as input.
2. Separate the sentence into individual words.
3. Assume the first word is the longest.
4. Compare each remaining word with the current longest word.
5. If a word has more characters, make it the new longest word.
6. Continue until every word has been checked.
7. Return the longest word.

10. Scenario: A plagiarism detection tool compares words from different documents and checks if they are anagrams (same characters but different order).

Write logic to check whether two given strings are anagrams.

Logic:

1. Take two strings as input.

2. Convert both strings to the same case if case should be ignored.
3. Remove spaces if spaces are not considered.
4. Check whether both strings contain the same number of characters.
5. Compare the frequency of every character in both strings.
6. If every character has the same frequency, the strings are anagrams.
7. Otherwise, they are not anagrams.